

Digitale Systeme 2025 - Programmierprojekt

Ihre Aufgabe besteht einerseits darin, die empfangenen Zeichen einer bitseriellen Datenübertragung eines gegebenen Mikrocontroller-Simulators zu deserialisieren und darzustellen. Andererseits sollen Sie Software für den Mikrocontroller-Simulator entwickeln. Die Programmieraufgabe besteht somit aus zwei Teilen.

Vorbereitung

- Laden Sie sich die Datei **projekt.tgz** von der Moodle-Seite des Kurses herunter und legen Sie diese auf einem gruenau-Rechner in einem Verzeichnis Ihrer Wahl ab. Dieses Verzeichnis wird der Einfachheit halber im Folgenden D genannt. Für den ersten Teil der Aufgabe können Sie auch einen Linux-Rechner verwenden, auf dem die Programme make und die Compiler gcc und g++ installiert sind.
- Führen Sie innerhalb D folgenden Befehl auf der Kommandozeile aus:

```
tar -xf projekt.tgz
```

Nach erfolgter Ausführung liegt innerhalb von D das entpackte Verzeichnis **projekt**.

- D/**projekt** ist nun Ihr Arbeitsverzeichnis. Alle folgenden Instruktionen bzgl. der Ausführung von Befehlen gehen davon aus, dass Sie sich in diesem Verzeichnis befinden. Wir empfehlen Ihnen daher, dieses Verzeichnis mit einer IDE Ihrer Wahl zu öffnen. Außerdem sollten Sie ein Terminal öffnen und in das Arbeitsverzeichnis navigieren, um dort Befehle ausführen zu können.

Teil eins: Vervollständigung des Simulators

Der erste Teil besteht darin, ein C-Programm zu schreiben, um die seriellen Bits, die aus dem Simulator kommen, wieder zu parallelisieren, um sie anschließend auf der Standardausgabe auszugeben.

Eigenschaften des Simulators

Der gesamte Simulator ist im Verzeichnis `./obj_dir` untergebracht und ist in der Programmiersprache C++ beschrieben. Ändern Sie nichts an den Quelldateien des Simulators, weil wir sonst Ihre Abgabe nicht bewerten können. Die Bestandteile des Simulators sind ein kleiner Mikrocontroller mit einem RISC-V-Prozessor und einer seriellen Schnittstelle.

In Abbildung 1 sind die Eigenschaften der seriellen Schnittstelle zusammengefasst. Wie viele Taktzyklen der Simulator ausführt, ist mit dem Bezeichner „MAX_SIM_TIME“ in der Datei „tb_TopLevel.cpp“ definiert. Den Simulator können Sie übersetzen und ausführen, indem Sie das Shellsript „./run.sh“ aufrufen

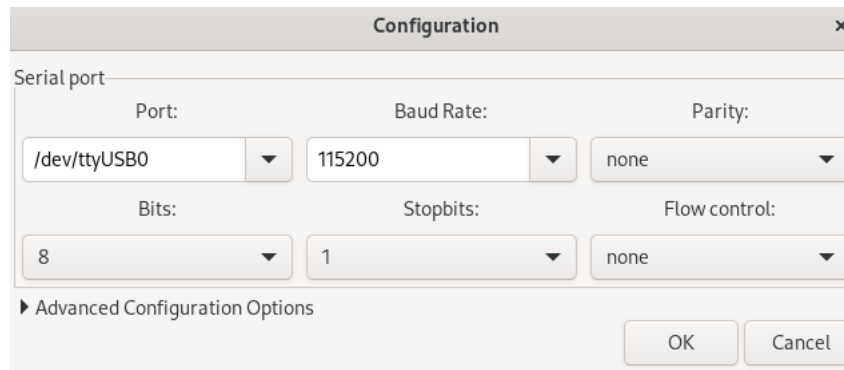


Abbildung 1: Eigenschaften der seriellen Schnittstelle

Der Prozessor wird mit einer Taktgeschwindigkeit von 90 MHz betrieben, wobei der Simulator seinen neuen Zustand mit der steigenden und fallenden Taktflanke neu berechnet. Der RISC-V-Prozessor ist ein 32-Bit-Rechner, der keine Standarderweiterung (rv32i) besitzt. Dies bedeutet, dass es keine Maschinenbefehle für Floating Point- oder Multiplikations- bzw. Divisionensoperationen gibt.

Vorgehen

Mit jedem Simulationszyklus wird die Funktion „void printOut(uint8_t val)“ aufgerufen. Die Funktion „void printOut(uint8_t val)“ ist definiert in der Datei „./src/rs232.c“. Für die Lösung des ersten Teils der Aufgabe arbeiten Sie ausschließlich in der „rs232.c“ Datei. Das ist die Datei, die Sie als Lösung für den ersten Teil der Programmieraufgabe einreichen. Die Variable „val“ stellt den Ausgang der seriellen Schnittstelle des Prozessors dar (TX). „val“ kann den Wert '0' oder '1' einnehmen. Berechnen Sie den Wert des Taktteilers und tasten Sie die Variable „val“ korrekt ab. Wenn die serielle Übertragung eines Zeichens vollendet ist, geben Sie das Zeichen auf der Standardausgabe mit der Funktion printf() aus. Wenn Sie die seriellen Bits korrekt parallelisiert und ausgeben, muss Ihre Ausgabe wie in Bild 2 aussehen.

Abgabe

Ihre komplette Lösung für den ersten Teil der Aufgabe soll in der rs232.c Datei von Ihnen programmiert werden. Die Verwendung weiterer Quelldateien in Abgaben ist nicht zulässig und steht nicht zur Diskussion. Weiterhin dürfen ausschließlich die in der Aufgabenstellung bereits inkludierten Header verwendet werden, die Verwendung weiterer

Header ist nicht zulässig. Das Programm soll mit dem Shellsript „./run.sh“ übersetzt werden können. Als Referenzsystem dient **gruenau4.informatik.hu-berlin.de**. Auf diesem Rechner muss Ihr Code funktionieren. Abgaben, die dort nicht kompilieren oder dort nicht korrekt ausgeführt werden können, werden als nicht bestanden gewertet.

```
RISC-V Prozessor der
Humboldt Universitaet zu Berlin
Counter:0 - Variable 'c' as int: 48 and variable 'c' as char: 0
Counter:1 - Variable 'c' as int: 49 and variable 'c' as char: 1
Counter:2 - Variable 'c' as int: 50 and variable 'c' as char: 2
Counter:3 - Variable 'c' as int: 51 and variable 'c' as char: 3
Counter:4 - Variable 'c' as int: 52 and variable 'c' as char: 4
Counter:5 - Variable 'c' as int: 53 and variable 'c' as char: 5
Counter:6 - Variable 'c' as int: 54 and variable 'c' as char: 6
Counter:7 - Variable 'c' as int: 55 and variable 'c' as char: 7
Counter:8 - Variable 'c' as int: 56 and variable 'c' as char: 8
Counter:9 - Variable 'c' as int: 57 and variable 'c' as char: 9
Counter:10 - Variable 'c' as int: 58 and variable 'c' as char: :
Counter:11 - Variable 'c' as int: 59 and variable 'c' as char: ;
Counter:12 - Variable 'c' as int: 60 and variable 'c' as char: <
Counter:13 - Variable 'c' as int: 61 and variable 'c' as char: =
Counter:14 - Variable 'c' as int: 62 and variable 'c' as char: >
Counter:15 - Variable 'c' as int: 63 and variable 'c' as char: ?
```

Abbildung 2: Korrekte Ausgabe nach der Ausführung des Simulators

Teil zwei: Schreiben Sie ein Bare-Metal-RISC-V (rv32i)-Programm für den Simulator, um den Arcustangens (atan2) in Festkommaarithmetik auszurechnen

Ein Sensor hat seine Messungen in den Datenspeicher geschrieben. Ihre Aufgabe ist es, einen Teil der Sensordatenverarbeitung mit unserem Mikrocontroller vorzunehmen. Bei dem Sensor handelt es sich um eine „Time-of-Flight“-3D-Kamera, die für jeden Pixel vier Messungen vornimmt. Im Gegensatz zu herkömmlichen Kameras gibt jeder Pixel einer ToF-Kamera die Distanz zu einem Objekt an. Das Resultat ist somit eine Tiefenkarte und keine zweidimensionale Abbildung der Szene.

Das Pixelfeld in unserem Beispiel hat eine Größe von 8 x 8 Pixel. Es werden somit pro Aufnahme 256 Werte in den Speicher geschrieben (8 x 8 x 4). Die Reihenfolge, in der die Werte in den Speicher geschrieben werden, ist fortlaufend. Der Offset für die Speicheradressen der ersten vier Werte des ersten Pixels sind somit [0,1,2,3], für das zweite Pixel [4,5,6,7] und so weiter. Die Anfangsadresse der Sensordaten ist in dem Pointer „tof“ gespeichert. Aus den vier Messwerten wird bei ToF-Kameras die Distanz mit der Formel: $D_{tof}[m] = \frac{c}{2 \cdot 2\pi f_{LED}} [\pi + atan2(\frac{a_3 - a_1}{a_2 - a_0})] + D_{OFFSET}$ berechnet. Weil unser RISC-V-Prozessor keine Floating-Point-Einheit besitzt, sind die Sensordaten in einem

Fixpointformat mit 32 Bit in den Speicher geschrieben worden, der Präzisionspunkt ist bei dem Bit 17.

Vorgehen

Laden Sie sich das Archiv für den zweiten Teil der Aufgabe herunter und entpacken Sie es in den Ordner `projekt/`. Die Dateien, die Sie für Ihre Lösung bearbeiten müssen, sind die:

`main_rv32.c`

`fptc.h`

Beide Dateien sind in dem Unterordner `rv32iProgramm/`.

Bevor Sie das Programm übersetzen, müssen Sie den RISC-V gcc Compiler dem System bekannt machen. Das geschieht, indem Sie auf einem gruenau-Server den Befehl `module load riscv/rv32i` im Terminal eingeben. Das RISC-V Programm übersetzen Sie, indem Sie im Ordner `rv32iProgramm` den Befehl `make` ausführen. Der Befehl `make clean` entfernt die ausführbaren Dateien in dem Programmverzeichnis.

Verwenden Sie für Ihre Lösung die fixed point numeric library. Dafür gehen Sie auf die GitHub-Seite „<https://github.com/mgetka/fptc-lib>“ und fügen den Inhalt der Headerdatei `fptc.h` in die entsprechende Datei im `rv32iProgramm`-Ordner ein. Damit die Bibliothek für die Festkommaarithmetik in Bare Metal funktioniert, muss eine geringfügige Anpassung vorgenommen werden.

Die Arctan2 Funktion implementieren Sie in der `main_rv32.c` Datei. Diese Datei geben Sie als Lösung für den zweiten Teil der Aufgabe ab. Es reicht, wenn Sie die geforderte Arctan2-Funktion näherungsweise ausrechnen. Sie sollen nicht die vollständige Formel für die Tiefenwertberechnung implementieren, sondern lediglich den Teil mit der Arctan2-Funktion.

$$D_{tof}[m] = atan2\left(\frac{a_3 - a_1}{a_2 - a_0}\right)$$

Abschließend müssen die errechneten Werte in Form einer 8×8 -Matrix auf die Standardausgabe geschrieben werden. Wenn der Simulator mit Ihrem Bare-Metal-RISC-V (rv32i)-Programm ausgeführt wird, soll die Ausgabe wie in Abbildung 3 aussehen. Ihr Ergebnis sollte auf zwei Nachkommastellen genau sein.

Gegebenenfalls benötigt Ihr Simulator eine etwas längere Zeit als voreingestellt, um alle 64 Werte zu berechnen. Um die Simulationszeit zu verlängern, können Sie in der Datei `projekt/src/tb_TopLevel.cpp` die Konstante `MAX_SIM_TIME` anpassen. `MAX_SIM_TIME` gibt die Anzahl der Taktzyklen an, die simuliert werden.

Abgabe

Ihre komplette Lösung für den zweiten Teil der Aufgabe soll in der `main_rv32.c` Datei von Ihnen programmiert werden. Die Verwendung weiterer Quelldateien in Abgaben ist nicht zulässig und steht nicht zur Diskussion. Weiterhin dürfen ausschließlich die in der Aufgabenstellung bereits inkludierten Header verwendet werden, die Verwendung

weiterer Header ist nicht zulässig. Das Programm soll mit dem Aufruf von make übersetzt werden können. Als Referenzsystem dient **gruenau4.informatik.hu-berlin.de**. Auf diesem Rechner muss Ihr Code funktionieren. Abgaben, die dort nicht kompilieren oder dort nicht korrekt ausgeführt werden können, werden als nicht bestanden gewertet.

Viel Erfolg!

```
| 0.723692 | 1.695220 | -1.314009 | 0.292804 | 1.606219 | -3.103330 | -2.944634 | -2.178651 |
| -3.060289 | -2.772990 | 1.526554 | 2.024808 | 0.878879 | -2.264897 | 0.622960 | -0.385232 |
| -0.999259 | 2.011042 | -2.670629 | 2.469066 | 0.371567 | -2.677865 | -2.884543 | -2.711031 |
| -1.833132 | 2.237077 | 1.535606 | -2.768218 | -1.478809 | -1.561405 | -2.775122 | 2.626833 |
| 1.601317 | 2.832616 | 0.704438 | 1.456709 | 0.184254 | -0.449568 | 2.766616 | -2.619220 |
| -1.354919 | 1.585080 | 1.811674 | 0.084834 | -0.800687 | -2.965947 | 0.474443 | -2.128577 |
| 1.459959 | 0.588057 | 3.076115 | 3.137887 | -0.277652 | -2.517128 | -0.290933 | -3.024814 |
| 2.860610 | -2.885426 | -0.171662 | 3.051290 | 2.241831 | 1.727061 | 2.687083 | -1.449298 |
```

Abbildung 3: Korrekte Ausgabe nach der Ausführung des zweiten Teils